

# **STUDENT EXPENSE MANAGER**

**Python & SQLite Project Report**

***VITyarthi – Build Your Own Project***

Prepared for submission

# 1. Introduction

Student Expense Manager is a command-line Python application that helps students record and analyze daily expenses. It uses SQLite for persistent local storage and a modular Python architecture.

## 2. Problem Statement

Students often track spending informally, making it difficult to review total spending or identify high-spending categories. The system provides structured storage and analysis.

## 3. Objectives

Implement expense CRUD, input validation, SQLite persistence, category/month summaries, report generation, modular Python programming and unit testing.

## 4. Functional Requirements

The system provides Expense CRUD, validation/error handling, analytics and report generation. Inputs include title, amount, category, date and notes; outputs include records, totals, summaries and reports.

## 5. Non-Functional Requirements

Usability, reliability, maintainability, error handling, performance and resource efficiency are addressed through a menu interface, transactions, modular code, validation and standard-library-only implementation.

## 6. System Architecture

The CLI handles interaction; validators check input; the database module handles SQLite; services implement calculations; reports format analytical results. Diagram source is in docs/architecture.md.

## 7. Design Diagrams

Architecture, workflow, use-case, class/component, sequence and ER diagrams are supplied in Mermaid format in docs/architecture.md.

## 8. Design Decisions & Rationale

SQLite was selected because it is included with Python and needs no server. Modules separate database access, validation, business logic and presentation.

## 9. Implementation Details

The project uses sqlite3, datetime, collections and unittest. Database operations use parameterized SQL. Validation checks required fields, positive amounts and YYYY-MM-DD dates.

## 10. Screenshots / Results

The included demo script creates sample expenses, displays records, shows analytics and generates a report. Run the program locally to capture screenshots if required by the portal.

## 11. Testing Approach

Unit tests cover validation, total calculation and category aggregation. Run `python -m unittest discover -s tests -v`.

## 12. Challenges Faced

Handling invalid input, separating database logic from business logic and presenting analytics simply were addressed through validation and modular design.

## **13. Learnings & Key Takeaways**

The project demonstrates Python modules, functions, exception handling, SQLite CRUD, aggregation, unit testing, documentation and basic software architecture.

## **14. Future Enhancements**

Possible improvements include a graphical interface, login, budget alerts, CSV export, charts, recurring expenses and cloud synchronization.

## **15. References**

Python standard-library concepts: sqlite3, unittest, datetime and collections. Project requirements are based on the supplied VITyarthi Build Your Own Project instructions.